

# TP3

## Administration Cloud et Automatisation Azure

Azure CLI - Bash - Python - Monitoring - Exploitation

**Bloc 4 - Optimisation du SI par l'apport du Cloud Computing**

Support de travaux pratiques - niveau Mastère

Version Azure

|                            |                                                              |
|----------------------------|--------------------------------------------------------------|
| <b>Parcours</b>            | Mastère Dev Manager Full Stack                               |
| <b>Positionnement</b>      | Suite du TP1 Azure et du TP2 Terraform Azure                 |
| <b>Fil rouge</b>           | Exploitation de l'environnement ShopEasy                     |
| <b>Production attendue</b> | Scripts, captures, rapport d'exploitation et recommandations |

## Table des matières

|           |                                                               |           |
|-----------|---------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Finalité du TP</b>                                         | <b>3</b>  |
| 1.1       | Compétences principalement travaillées                        | 3         |
| 1.2       | Objectifs pédagogiques                                        | 3         |
| 1.3       | Prérequis                                                     | 3         |
| <b>2</b>  | <b>Contexte fil rouge : exploitation de ShopEasy</b>          | <b>4</b>  |
| 2.1       | Environnement cible                                           | 4         |
| 2.2       | Nom des variables utilisées dans le TP                        | 4         |
| <b>3</b>  | <b>Rappels théoriques courts</b>                              | <b>4</b>  |
| 3.1       | Administration cloud                                          | 4         |
| 3.2       | Différence entre portail, CLI, script et IaC                  | 4         |
| 3.3       | Notion de runbook                                             | 5         |
| <b>4</b>  | <b>Atelier 1 - Prise en main d’Azure CLI</b>                  | <b>5</b>  |
| 4.1       | Objectif                                                      | 5         |
| 4.2       | Commandes de base                                             | 5         |
| 4.3       | Travail demandé                                               | 5         |
| <b>5</b>  | <b>Atelier 2 - Inventorier les ressources Azure</b>           | <b>6</b>  |
| 5.1       | Objectif                                                      | 6         |
| 5.2       | Lister les ressources                                         | 6         |
| 5.3       | Identifier les types de ressources                            | 6         |
| 5.4       | Travail demandé                                               | 6         |
| <b>6</b>  | <b>Atelier 3 - Normaliser les tags d’exploitation</b>         | <b>7</b>  |
| 6.1       | Objectif                                                      | 7         |
| 6.2       | Tags attendus                                                 | 7         |
| 6.3       | Appliquer des tags sur le groupe de ressources                | 7         |
| 6.4       | Appliquer un tag sur une VM                                   | 7         |
| 6.5       | Travail demandé                                               | 7         |
| <b>7</b>  | <b>Atelier 4 - Administrer les machines virtuelles</b>        | <b>8</b>  |
| 7.1       | Objectif                                                      | 8         |
| 7.2       | Lister les VM et leurs états                                  | 8         |
| 7.3       | Démarrer, redémarrer, arrêter                                 | 8         |
| 7.4       | Exécuter une commande distante                                | 8         |
| 7.5       | Travail demandé                                               | 8         |
| <b>8</b>  | <b>Atelier 5 - Construire un script Bash d’inventaire</b>     | <b>9</b>  |
| 8.1       | Objectif                                                      | 9         |
| 8.2       | Script attendu : <code>inventory.sh</code>                    | 9         |
| 8.3       | Travail demandé                                               | 9         |
| <b>9</b>  | <b>Atelier 6 - Automatiser l’arrêt et le démarrage des VM</b> | <b>10</b> |
| 9.1       | Objectif                                                      | 10        |
| 9.2       | Script attendu : <code>vm-power.sh</code>                     | 10        |
| 9.3       | Travail demandé                                               | 11        |
| <b>10</b> | <b>Atelier 7 - Exploiter un Storage Account</b>               | <b>11</b> |
| 10.1      | Objectif                                                      | 11        |
| 10.2      | Vérifier le compte de stockage                                | 11        |
| 10.3      | Travail demandé                                               | 12        |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>11 Atelier 8 - Surveiller les métriques avec Azure Monitor</b> | <b>12</b> |
| 11.1 Objectif . . . . .                                           | 12        |
| 11.2 Identifier l'ID de la VM . . . . .                           | 12        |
| 11.3 Lister les métriques disponibles . . . . .                   | 12        |
| 11.4 Lire la métrique CPU . . . . .                               | 12        |
| 11.5 Créer une alerte CPU . . . . .                               | 12        |
| 11.6 Travail demandé . . . . .                                    | 13        |
| <b>12 Atelier 9 - Ecrire un script de contrôle de santé</b>       | <b>13</b> |
| 12.1 Objectif . . . . .                                           | 13        |
| 12.2 Script attendu : <code>healthcheck.sh</code> . . . . .       | 13        |
| 12.3 Travail demandé . . . . .                                    | 14        |
| <b>13 Atelier 10 - Automatiser avec Python et Azure SDK</b>       | <b>14</b> |
| 13.1 Objectif . . . . .                                           | 14        |
| 13.2 Installation des dépendances . . . . .                       | 14        |
| 13.3 Script Python : <code>inventory.py</code> . . . . .          | 15        |
| 13.4 Travail demandé . . . . .                                    | 15        |
| <b>14 Atelier 11 - Analyse FinOps d'exploitation</b>              | <b>15</b> |
| 14.1 Objectif . . . . .                                           | 15        |
| 14.2 Actions à analyser . . . . .                                 | 16        |
| 14.3 Travail demandé . . . . .                                    | 16        |
| <b>15 Atelier 12 - Rapport d'exploitation ShopEasy</b>            | <b>16</b> |
| 15.1 Objectif . . . . .                                           | 16        |
| 15.2 Structure attendue . . . . .                                 | 16        |
| 15.3 Tableau de synthèse attendu . . . . .                        | 17        |
| <b>16 Mission autonome de fin de TP</b>                           | <b>17</b> |
| 16.1 Production attendue . . . . .                                | 17        |
| 16.2 Contraintes . . . . .                                        | 17        |
| <b>17 Livrables à rendre</b>                                      | <b>18</b> |
| <b>18 Quiz final</b>                                              | <b>18</b> |
| 18.1 Questions . . . . .                                          | 18        |
| <b>19 Grille d'évaluation indicative</b>                          | <b>18</b> |
| <b>20 Corrigé indicatif</b>                                       | <b>19</b> |
| 20.1 Inventaire . . . . .                                         | 19        |
| 20.2 Gestion des VM . . . . .                                     | 19        |
| 20.3 Monitoring . . . . .                                         | 19        |
| 20.4 Sécurité . . . . .                                           | 19        |
| 20.5 FinOps . . . . .                                             | 19        |
| <b>21 Annexe - Aide mémoire Azure CLI</b>                         | <b>19</b> |
| <b>22 Annexe - Ressources documentaires</b>                       | <b>20</b> |

## Finalité du TP

Ce TP met les apprenants en situation d'exploitation d'une infrastructure Azure déjà conçue et partiellement automatisée. L'objectif n'est plus seulement de créer des ressources, mais de les administrer, les inventorier, les surveiller, les automatiser et produire une documentation exploitable par une équipe d'exploitation.

## Compétences principalement travaillées

|            |                                                                                                                       |
|------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>C23</b> | Administrer et optimiser les infrastructures cloud avec des commandes Unix, scripts Bash/Shell et programmes Python.  |
| <b>C24</b> | Analyser et optimiser la performance des systèmes cloud avec des outils de monitoring, des métriques et des alertes.  |
| <b>C21</b> | Prendre en compte le coût et l'efficacité opérationnelle dans les choix d'administration et d'automatisation.         |
| <b>C25</b> | Appliquer des règles de sécurité d'exploitation : accès limités, audit, surveillance, contrôle des ouvertures réseau. |

## Objectifs pédagogiques

A l'issue de ce TP, l'apprenant devra être capable de :

- utiliser Azure CLI pour administrer un environnement cloud ;
- inventorier les ressources Azure d'un groupe de ressources ;
- manipuler les sorties JSON, table et TSV ;
- écrire des scripts Bash d'exploitation ;
- automatiser des actions courantes sur des machines virtuelles ;
- interroger les ressources Azure avec Python ;
- mettre en place une supervision simple avec Azure Monitor ;
- produire un rapport d'exploitation lisible pour une équipe technique ;
- proposer des pistes d'optimisation en coût, sécurité et maintenabilité.

## Prérequis

- Avoir réalisé le TP1 Azure ou disposer d'un environnement Azure équivalent.
- Avoir réalisé le TP2 Azure ou disposer d'un groupe de ressources contenant au moins une VNet, des VM, un compte de stockage et des règles réseau.
- Disposer d'un accès Azure avec les droits nécessaires sur le groupe de ressources.
- Avoir accès à Azure Cloud Shell ou à un poste avec Azure CLI installé.
- Savoir lire une commande Bash simple.

### Point de vigilance

Toutes les manipulations doivent être réalisées dans un groupe de ressources dédié au TP. Les ressources créées doivent être supprimées ou désallouées à la fin pour éviter les coûts inutiles. Ne jamais exécuter les commandes sur un environnement de production.

## Contexte fil rouge : exploitation de ShopEasy

ShopEasy a commencé sa migration vers Azure. L'équipe projet a conçu une architecture cible au TP1 et a commencé à déployer des ressources avec Terraform au TP2. La direction technique souhaite maintenant structurer l'exploitation quotidienne : inventaire, suivi des ressources, opérations courantes, surveillance, alertes et rapport d'exploitation.

### Environnement cible

L'environnement à exploiter contient idéalement :

- un groupe de ressources dédié, par exemple `rg-shopeasy-dev` ;
- un Virtual Network et au moins un subnet ;
- un Network Security Group ;
- une ou deux machines virtuelles Linux ;
- un Load Balancer ou une exposition HTTP simple ;
- un Storage Account ;
- éventuellement une base managée ou un service de supervision.

Si certaines ressources n'existent pas dans votre environnement, le formateur peut fournir un environnement préparé ou demander aux apprenants d'adapter les scripts aux ressources disponibles.

### Nom des variables utilisées dans le TP

Les commandes suivantes supposent les variables ci-dessous. Elles sont à adapter à votre environnement.

```
export RG="rg-shopeasy-dev"
export LOCATION="francecentral"
export VM1="vm-shopeasy-web-01"
export VM2="vm-shopeasy-web-02"
export STORAGE="stshopeasydev001"
export CONTAINER="operations"
```

#### Validation attendue

Les apprenants doivent conserver un fichier `variables.sh` contenant les variables utilisées pendant le TP. Ce fichier sera inclus dans les livrables.

## Rappels théoriques courts

### Administration cloud

Administrer une infrastructure cloud consiste à réaliser les opérations nécessaires pour maintenir un environnement fonctionnel, sécurisé, observable et maîtrisé en coût. Contrairement à une approche purement manuelle via le portail, l'administration cloud moderne repose fortement sur l'automatisation.

### Différence entre portail, CLI, script et IaC

| Approche      | Usage principal                                | Limites                                     |
|---------------|------------------------------------------------|---------------------------------------------|
| Portail Azure | Découverte, visualisation, actions ponctuelles | Peu reproductible, risque d'erreur manuelle |

|                     |                                                       |                                                        |
|---------------------|-------------------------------------------------------|--------------------------------------------------------|
| Azure CLI           | Administration rapide et commandes scriptables        | Nécessite de comprendre les paramètres et sorties      |
| Scripts Bash/Python | Automatisation de tâches récurrentes                  | Doivent être testés, versionnés et documentés          |
| Terraform / IaC     | Création et évolution déclarative de l'infrastructure | Moins adapté aux opérations ponctuelles d'exploitation |
| Azure Monitor       | Surveillance, métriques, logs, alertes                | Nécessite de bons seuils et une interprétation métier  |

## Notion de runbook

Un runbook est une procédure d'exploitation. Il décrit comment réaliser une action de manière fiable : démarrer une application, arrêter un environnement, vérifier la santé d'un service, restaurer un fichier, analyser une alerte ou produire un rapport.

Dans ce TP, les apprenants construisent progressivement un mini-runbook d'exploitation pour ShopEasy.

## Atelier 1 - Prise en main d'Azure CLI

### Objectif

Vérifier l'accès à l'environnement Azure, identifier la subscription active et se positionner sur le bon groupe de ressources.

### Commandes de base

```
az version
az login
az account show --output table
az account list --output table
```

Si plusieurs subscriptions sont disponibles :

```
az account set --subscription "<ID_OU_NOM_SUBSCRIPTION>"
az account show --output table
```

Lister les régions Azure disponibles :

```
az account list-locations --query "[].{Nom:name,DisplayName:displayName}" --output table
```

Lister les groupes de ressources :

```
az group list --output table
az group show --name $RG --output json
```

### Travail demandé

1. Vérifier la subscription active.
2. Vérifier que le groupe de ressources du TP existe.
3. Configurer les valeurs par défaut pour éviter de répéter les mêmes paramètres.

```
az configure --defaults group=$RG location=$LOCATION
az configure --list-defaults
```

## Livable attendu

Capture ou extrait de terminal montrant : subscription active, groupe de ressources cible et valeurs par défaut configurées.

## Atelier 2 - Inventorier les ressources Azure

## Objectif

Produire un inventaire lisible des ressources ShopEasy. Cet inventaire doit pouvoir être exécuté à nouveau sans passer par le portail.

## Lister les ressources

```
az resource list --resource-group $RG --output table
az resource list --resource-group $RG --output json
```

Afficher seulement les colonnes utiles :

```
az resource list \
--resource-group $RG \
--query "[].{Nom:name,Type:type,Region:location}" \
--output table
```

Exporter l'inventaire en JSON :

```
mkdir -p exports
az resource list --resource-group $RG --output json > exports/resources.json
```

Exporter l'inventaire en TSV :

```
az resource list \
--resource-group $RG \
--query "[].{Nom:name,Type:type,Region:location}" \
--output tsv > exports/resources.tsv
```

## Identifier les types de ressources

```
az resource list \
  --resource-group $RG \
  --query "[].type" \
  --output tsv | sort | uniq -c
```

## Travail demandé

Créer un tableau d'inventaire.

| Nom ressource | Type Azure | Région | Rôle dans l'architecture |
|---------------|------------|--------|--------------------------|
|---------------|------------|--------|--------------------------|

**Livrable attendu**

Fichiers attendus : `exports/resources.json`, `exports/resources.tsv` et tableau d'inventaire complété dans le rapport d'exploitation.

## Atelier 3 - Normaliser les tags d'exploitation

### Objectif

Appliquer une stratégie de tags minimale pour faciliter l'exploitation, le suivi des coûts et l'identification des responsabilités.

### Tags attendus

|                    |                                      |
|--------------------|--------------------------------------|
| <b>Application</b> | shopeasy                             |
| <b>Environment</b> | dev, test ou prod selon le contexte  |
| <b>Owner</b>       | équipe ou responsable                |
| <b>CostCenter</b>  | centre de coût ou valeur pédagogique |
| <b>ManagedBy</b>   | cli, terraform, portal ou mixed      |

### Appliquer des tags sur le groupe de ressources

```
az group update \  
  --name $RG \  
  --tags Application=shopeasy Environment=dev Owner=formation CostCenter=cloud-lab ManagedBy=cli
```

Vérifier :

```
az group show --name $RG --query tags --output table
```

### Appliquer un tag sur une VM

```
az vm update \  
  --resource-group $RG \  
  --name $VM1 \  
  --set tags.Application=shopeasy tags.Environment=dev tags.Role=web
```

### Travail demandé

1. Appliquer les tags au groupe de ressources.
2. Appliquer au moins trois tags aux VM.
3. Vérifier que les tags sont visibles dans le portail et avec Azure CLI.
4. Expliquer pourquoi les tags sont utiles pour le FinOps et la gouvernance.

**Livrable attendu**

Capture ou export JSON montrant les tags du groupe de ressources et d'au moins une VM.



## Atelier 4 - Administrer les machines virtuelles

### Objectif

Réaliser les opérations courantes sur les VM : lister, vérifier l'état, démarrer, arrêter, désallouer, redémarrer et exécuter une commande distante.

### Lister les VM et leurs états

```
az vm list --resource-group $RG --output table
az vm list --resource-group $RG --show-details --output table
```

Afficher un état synthétique :

```
az vm list \
  --resource-group $RG \
  --show-details \
  --query "[].{Nom:name,Etat:powerState,IP:publicIps,Taille:hardwareProfile.vmSize}" \
  --output table
```

### Démarrer, redémarrer, arrêter

```
az vm start --resource-group $RG --name $VM1
az vm restart --resource-group $RG --name $VM1
az vm stop --resource-group $RG --name $VM1
az vm deallocate --resource-group $RG --name $VM1
```

#### Point de vigilance

Dans Azure, arrêter une VM depuis le système d'exploitation ne suffit pas toujours à arrêter la facturation compute. Pour libérer les ressources compute, utiliser **az vm deallocate** ou l'arrêt depuis le portail avec l'état **Stopped** (deallocated).

### Exécuter une commande distante

Tester le service web depuis la VM :

```
az vm run-command invoke \
  --resource-group $RG \
  --name $VM1 \
  --command-id RunShellScript \
  --scripts "hostname && uptime && systemctl status nginx --no-pager || systemctl status apache2 --no-pager"
```

### Travail demandé

1. Lister toutes les VM du groupe de ressources.
2. Identifier leur taille, leur IP publique et leur état.
3. Redémarrer une VM.
4. Désallouer une VM non utilisée.
5. Exécuter une commande distante pour vérifier l'état du serveur web.

#### Livrable attendu

Tableau des VM avec état, IP, taille et action réalisée. Inclure la commande utilisée et le résultat observé.

## Atelier 5 - Construire un script Bash d'inventaire

### Objectif

Transformer les commandes manuelles en script réutilisable.

### Script attendu : `inventory.sh`

Créer un dossier `scripts` et un fichier `inventory.sh`.

```
mkdir -p scripts exports
nano scripts/inventory.sh
```

Contenu proposé :

```
#!/usr/bin/env bash
set -euo pipefail

RG="${1:-rg-shopeasy-dev}"
DATE=$(date +%Y%m%d-%H%M%S)
OUT_DIR="exports"
mkdir -p "$OUT_DIR"

echo "Inventaire Azure - groupe de ressources: $RG"
echo "Date: $DATE"

echo "Export des ressources..."
az resource list \
  --resource-group "$RG" \
  --query "[].{name:name,type:type,location:location,id:id}" \
  --output json > "$OUT_DIR/resources-$DATE.json"

echo "Export des VM..."
az vm list \
  --resource-group "$RG" \
  --show-details \
  --query "[].{name:name,powerState:powerState,publicIps:publicIps,vmSize:hardwareProfile.
    vmSize}" \
  --output table > "$OUT_DIR/vms-$DATE.txt"

echo "Export termine dans $OUT_DIR"
```

Rendre le script exécutable :

```
chmod +x scripts/inventory.sh
./scripts/inventory.sh $RG
```

### Travail demandé

Améliorer le script pour ajouter :

- le nombre total de ressources ;
- le nombre de VM ;
- le nombre de comptes de stockage ;
- le nombre de ressources sans tag `Application` ;
- un message d'avertissement si une VM est en état `VM running`.

### Livrable attendu

Script `inventory.sh` fonctionnel, captures d'exécution et fichiers générés dans `exports/`.

## Atelier 6 - Automatiser l'arrêt et le démarrage des VM

### Objectif

Construire un script simple permettant de démarrer ou désallouer les VM de l'environnement de développement.

### Script attendu : `vm-power.sh`

```
nano scripts/vm-power.sh
```

Contenu proposé :

```
#!/usr/bin/env bash
set -euo pipefail

RG="${1:-}"
ACTION="${2:-}"

if [[ -z "$RG" || -z "$ACTION" ]]; then
    echo "Usage: $0 <resource-group> <start|stop|deallocate|status>"
    exit 1
fi

VMS=$(az vm list --resource-group "$RG" --query "[] .name" --output tsv)

if [[ -z "$VMS" ]]; then
    echo "Aucune VM trouvee dans $RG"
    exit 0
fi

for VM in $VMS; do
    echo "Traitement de la VM: $VM"
    case "$ACTION" in
        start)
            az vm start --resource-group "$RG" --name "$VM"
            ;;
        stop)
            az vm stop --resource-group "$RG" --name "$VM"
            ;;
        deallocate)
            az vm deallocate --resource-group "$RG" --name "$VM"
            ;;
        status)
            az vm get-instance-view \
                --resource-group "$RG" \
                --name "$VM" \
                --query "instanceView.statuses[?starts_with(code, 'PowerState/')].displayStatus" \
                --output tsv
            ;;
        *)
            echo "Action inconnue: $ACTION"
            exit 1
            ;;
    esac
done
```

Tester :

```
chmod +x scripts/vm-power.sh
./scripts/vm-power.sh $RG status
```

```
./scripts/vm-power.sh $RG deallocate  
./scripts/vm-power.sh $RG start
```

## Travail demandé

1. Tester chaque action du script.
2. Ajouter une confirmation utilisateur avant `stop` ou `deallocate`.
3. Empêcher l'exécution sur un groupe de ressources dont le nom contient `prod`.
4. Ajouter une trace dans un fichier `logs/vm-power.log`.

### Livrable attendu

Script `vm-power.sh`, journal d'exécution et explication des mesures de sécurité ajoutées.

## Atelier 7 - Exploiter un Storage Account

### Objectif

Créer un espace de stockage opérationnel pour déposer des rapports d'exploitation et tester des opérations de sauvegarde simple.

### Vérifier le compte de stockage

```
az storage account show \  
--name $STORAGE \  
--resource-group $RG \  
--output table
```

Si aucun compte de stockage n'existe, le créer :

```
az storage account create \  
--name $STORAGE \  
--resource-group $RG \  
--location $LOCATION \  
--sku Standard_LRS \  
--kind StorageV2 \  
--allow-blob-public-access false
```

Créer un conteneur privé :

```
az storage container create \  
--account-name $STORAGE \  
--name $CONTAINER \  
--auth-mode login \  
--public-access off
```

Téléverser un rapport :

```
echo "Rapport exploitation ShopEasy" > exports/rapport-test.txt  
az storage blob upload \  
--account-name $STORAGE \  
--container-name $CONTAINER \  
--file exports/rapport-test.txt \  
--name rapports/rapport-test.txt \  
--auth-mode login \  
--overwrite true
```

Lister les fichiers :

```
az storage blob list \  
  --account-name $STORAGE \  
  --container-name $CONTAINER \  
  --auth-mode login \  
  --output table
```

## Travail demandé

1. Créer ou réutiliser un Storage Account sécurisé.
2. Créer un conteneur privé **operations**.
3. Déposer les exports d'inventaire dans ce conteneur.
4. Vérifier que l'accès public au compte de stockage est désactivé.
5. Expliquer pourquoi le stockage de rapports d'exploitation ne doit pas être public.

### Livrable attendu

Capture ou sortie CLI montrant le conteneur, les fichiers déposés et la configuration d'accès public désactivée.

## Atelier 8 - Surveiller les métriques avec Azure Monitor

### Objectif

Interroger les métriques Azure et créer une alerte simple sur une ressource critique.

### Identifier l'ID de la VM

```
VM_ID=$(az vm show --resource-group $RG --name $VM1 --query id --output tsv)  
echo $VM_ID
```

### Lister les métriques disponibles

```
az monitor metrics list-definitions \  
  --resource $VM_ID \  
  --output table
```

### Lire la métrique CPU

```
az monitor metrics list \  
  --resource $VM_ID \  
  --metric "Percentage CPU" \  
  --interval PT1M \  
  --aggregation Average \  
  --output table
```

### Créer une alerte CPU

Créer une règle d'alerte simple sans action group dans un premier temps :

```
az monitor metrics alert create \  
  --name "alert-cpu-high-$VM1" \  
  --resource-group $RG \  
  --scopes $VM_ID \  
  --condition "avg Percentage CPU > 80" \  
  --description "Alerte CPU elevee sur VM ShopEasy" \  
  --evaluation-frequency 1m \  
  --window-size 5m \  
  --severity 3
```

Lister les alertes :

```
az monitor metrics alert list --resource-group $RG --output table
```

## Travail demandé

1. Identifier au moins trois métriques disponibles pour une VM.
2. Lire la métrique CPU avec Azure CLI.
3. Créer une alerte CPU.
4. Proposer deux autres alertes utiles pour ShopEasy.
5. Expliquer les limites d'une alerte trop sensible ou trop large.

### Livrable attendu

Capture ou sortie CLI montrant la métrique CPU et la règle d'alerte créée. Inclure une justification du seuil choisi.

## Atelier 9 - Ecrire un script de contrôle de santé

### Objectif

Créer un script qui vérifie rapidement si l'environnement est dans un état acceptable.

### Script attendu : `healthcheck.sh`

```
nano scripts/healthcheck.sh
```

Contenu proposé :

```
#!/usr/bin/env bash  
set -euo pipefail  
  
RG="${1:-rg-shopeasy-dev}"  
STATUS=0  
  
echo "Controle de sante Azure - $RG"  
echo "-----"  
  
echo "1. Verification du groupe de ressources"  
if az group show --name "$RG" >/dev/null 2>&1; then  
  echo "OK - groupe de ressources trouve"  
else  
  echo "KO - groupe de ressources introuvable"  
  exit 1  
fi  
  
echo "2. Verification des VM"
```

```
az vm list --resource-group "$RG" --show-details \
--query "[].{name:name,state:powerState,ip:publicIps}" \
--output table

echo "3. Verification des ressources sans tag Application"
UNTAGGED=$(az resource list --resource-group "$RG" \
--query "[?tags.Application==null].name" --output tsv | wc -l)
if [[ "$UNTAGGED" -gt 0 ]]; then
    echo "WARN - $UNTAGGED ressource(s) sans tag Application"
    STATUS=1
else
    echo "OK - tags Application presents"
fi

echo "4. Verification des alertes Azure Monitor"
ALERTS=$(az monitor metrics alert list --resource-group "$RG" --query "length(@)" --output tsv
)
echo "Nombre d'alertes: $ALERTS"
if [[ "$ALERTS" -eq 0 ]]; then
    echo "WARN - aucune alerte configuree"
    STATUS=1
fi

exit $STATUS
```

Tester :

```
chmod +x scripts/healthcheck.sh
./scripts/healthcheck.sh $RG
```

## Travail demandé

Améliorer le script pour vérifier :

- l'existence du Storage Account ;
- l'existence du conteneur `operations` ;
- l'existence d'au moins une VM ;
- la présence d'un tag `Owner` ;
- l'existence d'au moins une règle NSG autorisant HTTP ou HTTPS.

### Livrable attendu

Script `healthcheck.sh` final, preuve d'exécution et commentaires sur les vérifications ajoutées.

## Atelier 10 - Automatiser avec Python et Azure SDK

### Objectif

Découvrir l'utilisation du SDK Azure pour Python afin de produire un inventaire programmable.

### Installation des dépendances

Dans un environnement Python local ou Cloud Shell :

```
python3 -m venv .venv
source .venv/bin/activate
pip install azure-identity azure-mgmt-resource azure-mgmt-compute
```

## Script Python : inventory.py

Créer un dossier python puis un fichier inventory.py.

```
import os
from azure.identity import DefaultAzureCredential
from azure.mgmt.resource import ResourceManagementClient
from azure.mgmt.compute import ComputeManagementClient

subscription_id = os.environ.get("AZURE_SUBSCRIPTION_ID")
resource_group = os.environ.get("AZURE_RESOURCE_GROUP", "rg-shopeasy-dev")

if not subscription_id:
    raise SystemExit("Variable AZURE_SUBSCRIPTION_ID manquante")

credential = DefaultAzureCredential()
resource_client = ResourceManagementClient(credential, subscription_id)
compute_client = ComputeManagementClient(credential, subscription_id)

print(f"Inventaire du groupe: {resource_group}")
print("Ressources:")
for res in resource_client.resources.list_by_resource_group(resource_group):
    print(f"- {res.name} | {res.type} | {res.location}")

print("\nMachines virtuelles:")
for vm in compute_client.virtual_machines.list(resource_group):
    size = vm.hardware_profile.vm_size
    print(f"- {vm.name} | taille={size} | region={vm.location}")
```

Exécuter :

```
export AZURE_SUBSCRIPTION_ID="<subscription-id>"
export AZURE_RESOURCE_GROUP=$RG
python python/inventory.py
```

## Travail demandé

Modifier le script Python pour produire un fichier CSV contenant :

- nom de la ressource ;
- type Azure ;
- région ;
- tags ;
- rôle supposé dans l'architecture.

### Livrable attendu

Script `inventory.py` et fichier CSV généré. Le CSV doit être lisible dans Excel ou LibreOffice Calc.

## Atelier 11 - Analyse FinOps d'exploitation

### Objectif

Identifier les actions simples permettant de limiter les coûts dans un environnement de développement.



## Actions à analyser

| Action                           | Effet attendu                 | Risque ou limite                     |
|----------------------------------|-------------------------------|--------------------------------------|
| Désallouer les VM hors usage     | Réduction du coût compute     | Indisponibilité de l'environnement   |
| Réduire la taille des VM         | Réduction du coût mensuel     | Performance plus faible              |
| Supprimer les disques inutilisés | Réduction du stockage facturé | Risque de supprimer une donnée utile |
| Standardiser les tags            | Meilleure analyse des coûts   | Discipline d'équipe nécessaire       |
| Définir un budget                | Alerte en cas de dérive       | Ne bloque pas toujours la dépense    |

## Travail demandé

1. Identifier les ressources susceptibles de générer le plus de coût.
2. Proposer une stratégie d'arrêt/démarrage des VM de développement.
3. Proposer une politique de tags FinOps.
4. Définir un seuil d'alerte budgétaire pertinent.
5. Rédiger trois recommandations FinOps pour la DSI.

### Livrable attendu

Tableau FinOps complété et recommandations courtes intégrées au rapport final.

## Atelier 12 - Rapport d'exploitation ShopEasy

### Objectif

Produire un rapport synthétique et exploitable, comme le ferait une équipe d'exploitation cloud.

### Structure attendue

Le rapport final doit contenir les sections suivantes :

1. Contexte et périmètre.
2. Inventaire des ressources.
3. Tags et gouvernance.
4. Etat des VM et actions réalisées.
5. Stockage opérationnel.
6. Monitoring et alertes.
7. Scripts produits.
8. Analyse sécurité.
9. Analyse FinOps.
10. Recommandations d'amélioration.

## Tableau de synthèse attendu

| Domaine    | Statut                            | Commentaire |
|------------|-----------------------------------|-------------|
| Inventaire | Conforme / partiel / non conforme |             |
| Tags       | Conforme / partiel / non conforme |             |
| VM         | Conforme / partiel / non conforme |             |
| Stockage   | Conforme / partiel / non conforme |             |
| Monitoring | Conforme / partiel / non conforme |             |
| Sécurité   | Conforme / partiel / non conforme |             |
| FinOps     | Conforme / partiel / non conforme |             |

### Livrable attendu

Un rapport PDF ou Markdown nommé **rapport-exploitation-shopeasy** accompagné des scripts et exports produits.

## Mission autonome de fin de TP

Vous êtes membre de l'équipe CloudOps de ShopEasy. Votre responsable vous demande de livrer un mini-kit d'exploitation permettant à une autre personne de reprendre l'administration de l'environnement sans dépendre du portail Azure.

## Production attendue

- un script `inventory.sh` ;
- un script `vm-power.sh` ;
- un script `healthcheck.sh` ;
- un script Python `inventory.py` ;
- un dossier `exports/` contenant les exports générés ;
- un rapport d'exploitation ;
- une synthèse des risques et recommandations.

## Contraintes

- Les scripts doivent être commentés.
- Les scripts doivent gérer les erreurs simples.
- Les scripts ne doivent pas exécuter d'action destructive sans confirmation.
- Les commandes doivent pouvoir être rejouées.
- Les résultats doivent être compréhensibles par un autre apprenant.

## Livrables à rendre

| Livrable               | Contenu attendu                           | Format                        |
|------------------------|-------------------------------------------|-------------------------------|
| Variables              | Fichier contenant les variables utilisées | <code>variables.sh</code>     |
| Scripts Bash           | Inventaire, gestion VM, contrôle de santé | <code>.sh</code>              |
| Script Python          | Inventaire via Azure SDK                  | <code>.py</code>              |
| Exports                | JSON, TSV, TXT, CSV selon les ateliers    | Dossier <code>exports/</code> |
| Captures               | Preuves d'exécution et ressources Azure   | PNG ou PDF                    |
| Rapport d'exploitation | Synthèse technique et recommandations     | PDF ou Mark-down              |
| Quiz                   | Réponses au quiz final                    | PDF ou texte                  |

## Quiz final

### Questions

1. Quelle est la différence entre `az vm stop` et `az vm deallocate` ?
2. Pourquoi est-il préférable d'utiliser Azure CLI plutôt que le portail pour une tâche répétitive ?
3. A quoi sert l'option `-query` dans Azure CLI ?
4. Quel format de sortie est utile pour réutiliser un résultat dans un script Bash ?
5. Pourquoi les tags sont-ils importants pour le FinOps ?
6. Qu'est-ce qu'un runbook d'exploitation ?
7. Pourquoi faut-il éviter d'ouvrir SSH à tout Internet ?
8. Quelle commande permet de lister les ressources d'un groupe de ressources ?
9. Quel service Azure permet de suivre les métriques et de créer des alertes ?
10. Quelle est la différence entre métrique et log ?
11. Pourquoi faut-il versionner les scripts d'exploitation ?
12. Quel est le rôle de `DefaultAzureCredential` dans un script Python Azure ?
13. Pourquoi faut-il tester un script sur un environnement non productif ?
14. Citer deux risques liés à un script d'administration mal sécurisé.
15. Quelle information doit figurer dans un rapport d'exploitation ?
16. Pourquoi un budget Azure ne remplace-t-il pas une gouvernance des ressources ?
17. Citer deux actions simples pour réduire les coûts d'un environnement de développement.
18. Pourquoi faut-il journaliser les actions d'un script ?
19. Quelle différence faites-vous entre IaC et script d'exploitation ?
20. Citer trois contrôles que vous intégreriez dans un `healthcheck.sh`.

## Grille d'évaluation indicative

| Critère | Points | Attendus |
|---------|--------|----------|
|---------|--------|----------|

|                                  |           |                                                            |
|----------------------------------|-----------|------------------------------------------------------------|
| Utilisation correcte d’Azure CLI | 3         | Commandes exécutées, sorties maîtrisées, bonnes variables  |
| Inventaire des ressources        | 3         | Exports lisibles, tableau complet, interprétation correcte |
| Scripts Bash                     | 4         | Scripts fonctionnels, robustes, commentés, non dangereux   |
| Script Python                    | 2         | Connexion Azure SDK, inventaire exploitable, export clair  |
| Monitoring et alertes            | 2         | Métriques lues, alerte créée, seuil justifié               |
| Analyse sécurité                 | 2         | Risques identifiés, mesures correctives pertinentes        |
| Analyse FinOps                   | 2         | Recommandations concrètes et adaptées                      |
| Rapport final                    | 2         | Structure claire, synthèse exploitable par une équipe      |
| <b>Total</b>                     | <b>20</b> |                                                            |

## Corrigé indicatif

### Inventaire

Un inventaire satisfaisant doit présenter au minimum : nom, type, région, tags principaux et rôle de chaque ressource. Les ressources non taguées doivent être signalées.

### Gestion des VM

Un environnement de développement ne doit pas conserver inutilement des VM en état **running**. Le script `vm-power.sh` doit privilégier `deallocate` pour réduire les coûts compute lorsque l’environnement n’est pas utilisé.

### Monitoring

Une alerte CPU seule ne suffit pas. Pour ShopEasy, on peut ajouter : disponibilité HTTP, espace disque, erreurs applicatives, disponibilité du Load Balancer, consommation budgétaire ou volume de requêtes.

### Sécurité

Les corrections minimales attendues sont : restriction de SSH, tags d’ownership, accès public désactivé sur le stockage, alerte active, journalisation des scripts, absence d’action destructive sans confirmation.

### FinOps

Les recommandations attendues incluent : désallocation planifiée des VM de développement, réduction de taille des VM, suppression des ressources orphelines, tags obligatoires et budget d’alerte.

## Annexe - Aide mémoire Azure CLI

```
az login
az account show -o table
az account set --subscription <id>
az group list -o table
az resource list -g <rg> -o table
```

```
az vm list -g <rg> --show-details -o table
az vm start -g <rg> -n <vm>
az vm deallocate -g <rg> -n <vm>
az monitor metrics list --resource <id> --
metric "Percentage CPU"
```

## Annexe - Ressources documentaires

- Documentation Azure CLI : <https://learn.microsoft.com/cli/azure/>
- Documentation Azure Monitor : <https://learn.microsoft.com/azure/azure-monitor/>
- Azure SDK for Python : <https://learn.microsoft.com/azure/developer/python/sdk/azure-sdk-overview>
- Cost Management : <https://learn.microsoft.com/azure/cost-management-billing/costs/>